DWH MCP Explorer

Stage 6.9.1 | Final System Architecture | Architecture baseline v1.0

Status: documented based on completed Stage 5–6 implementation and validated tests through 7 November 2026. This is an architecture snapshot, not a deployment-ready public repository.

1. Purpose and scope

DWH MCP Explorer is a custom Python Model Context Protocol (MCP) server enabling an AI client (Claude Desktop) to discover and interpret data-warehouse metadata, SQL definitions and lineage, compare mappings, and validate selected data-quality rules. Two backend capabilities coexist: a local PostgreSQL/SQL repository track and an Azure Databricks Lakehouse track.

2. High-level architecture

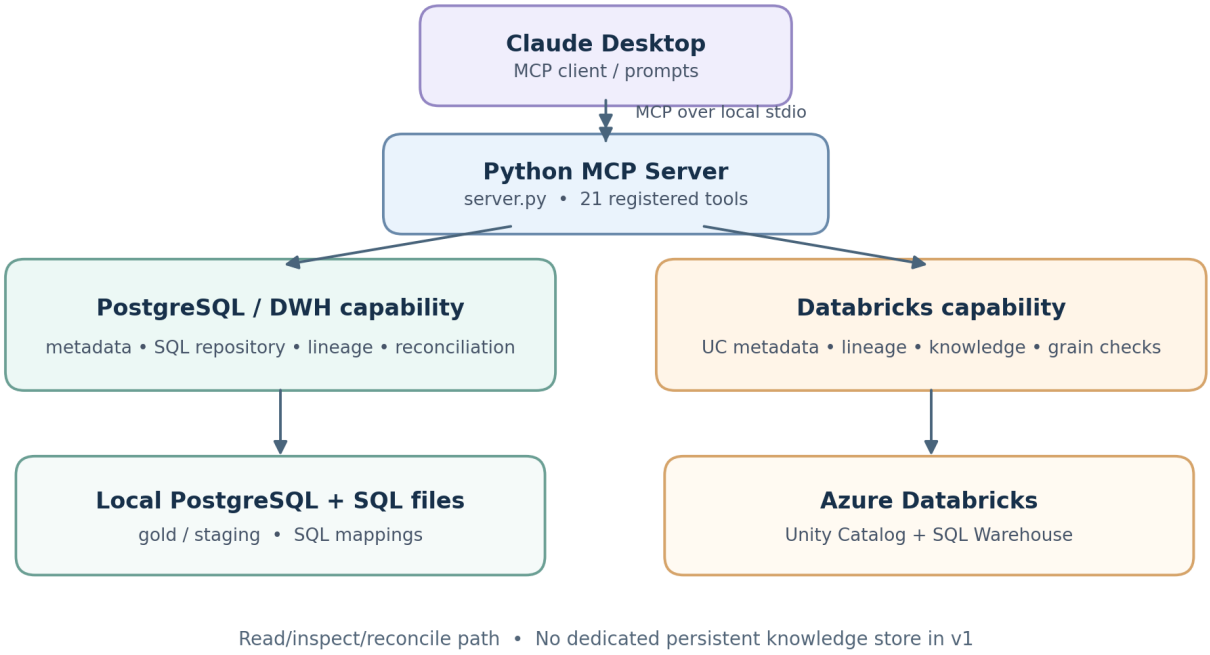


Figure 1. Logical architecture and principal data flows.

3. Core application components

Layer / module	Responsibility
Claude Desktop	MCP client: discovers tools, invokes operations and synthesizes technical reports.
server.py	Thin MCP adapter and tool registry; exposes 21 tools to the client.
dwh_mcp/database.py, sql_repository.py	PostgreSQL access and SQL file discovery/reading.
dwh_mcp/lineage.py, reconciliation.py, reporting.py, composition.py	Lineage extraction, structural/constraint reconciliation, output/report composition.

dwh_mcp/databricks_provider.py	Unity Catalog discovery: catalogs, schemas, tables, table metadata/columns.
dwh_mcp/databricks_lineage_provider.py	Queries system.access.table_lineage and system.access.column_lineage via the Databricks SQL Statement Execution API.
dwh_mcp/databricks_knowledge.py	Combines metadata + table/column lineage into one read-only, on-demand knowledge object.
dwh_mcp/databricks_reconciliation.py, databricks_grain_provider.py	Safe SQL generation and exact grain / business-key checks; converts SQL results into a validation report.

4. MCP tool inventory (21)

4.1 PostgreSQL / SQL / DWH tools (13)

list_schemas; list_tables; describe_table; get_primary_keys; get_foreign_keys; list_sql_files; read_sql_file; get_table_knowledge; get_table_lineage; reconcile_table_mapping; reconcile_column_constraints; generate_reconciliation_report; investigate_dwh_object.

4.2 Databricks tools (8)

databricks_list_catalogs; databricks_list_schemas; databricks_list_tables; databricks_describe_table; databricks_get_table_lineage; databricks_get_column_lineage; databricks_investigate_object; databricks_validate_grain.

5. Principal request flows

Flow	Sequence / output
Metadata investigation	Claude → MCP tool → Unity Catalog API or PostgreSQL metadata → structured response → AI explanation.
Databricks knowledge object	Claude → databricks_investigate_object → metadata provider + table lineage + column lineage → unified JSON (metadata, lineage, evidence, limitations).
Grain verification	Claude → databricks_validate_grain(full_name, key_column) → validated identifiers → read-only aggregate SQL through SQL Warehouse → total, distinct, null, duplicate-excess, pass/fail.
SQL/lineage reconciliation	Claude → PostgreSQL/SQL tools → metadata + SQL mapping / lineage → comparison/report output; rules limited to implemented checks.

6. Trust boundaries, operational controls and limitations

- Authentication: Databricks SDK uses a named CLI/OAuth profile. Credentials and refresh tokens are external to source code; local Claude configuration launches the Python MCP server.
- SQL safety: Grain validation accepts simple three-part identifiers and one simple key column, validates them, quotes identifiers and runs a read-only SELECT. It does not perform SQL transformations.
- Lineage coverage: Query history is bounded (currently 45-day lookback; 50 returned events per query). A non-truncated SDK result does not prove a complete dependency graph.
- Interpretation: Read-only events are not equivalent to confirmed source→target dependencies; column lineage does not include transformation expressions.
- Knowledge Base v1: On-demand composed JSON, not a durable indexed knowledge repository. execution_context is currently empty; entity IDs may be available within lineage events.

- Quality: Unity Catalog statistics may be approximate. Exact uniqueness/NULL findings require an executed aggregate query; successful test of gold.fact_orders returned 10,000 rows, 10,000 distinct order_id, 0 NULL and 0 duplicate excess.
- Operational caveat: Databricks SQL Warehouse queries can incur compute costs; OAuth credentials can expire and require login renewal.

7. Validation evidence and status

Validation group	Evidence / outcome
Stage 6.6 Knowledge Object	Metadata + table/column lineage composed; real Databricks test and Claude Desktop E2E confirmed.
Stage 6.7 Grain check	Provider mock tests, real SQL Warehouse validation, MCP client invocation and Claude Desktop E2E passed.
Stage 6.8 regression	7/7 recent regression tests; cross-platform registry; negative tests; MCP error flag; 5/5 legacy regression tests passed.
Registered tools	21 tools visible via MCP client; PostgreSQL/DWH and Databricks groups co-registered.

8. Explicitly out of scope / next documentation steps

Not implemented in this baseline: persistent search/index for the knowledge objects, complete lineage graph traversal, automated resolution of job/notebook names and SQL expressions, or full cross-platform reconciliation of every semantic/business rule. Stage 6.9.2–6.9.5 will elaborate tool contracts, test evidence, deployment/configuration and project closeout. A public GitHub release must separately audit configuration and secrets.